

НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Физический факультет

ОСНОВЫ ВЫЧИСЛИТЕЛЬНОЙ ФИЗИКИ

Лекции выжившего

Автор: БВО

Лектор: Усов Э. В.

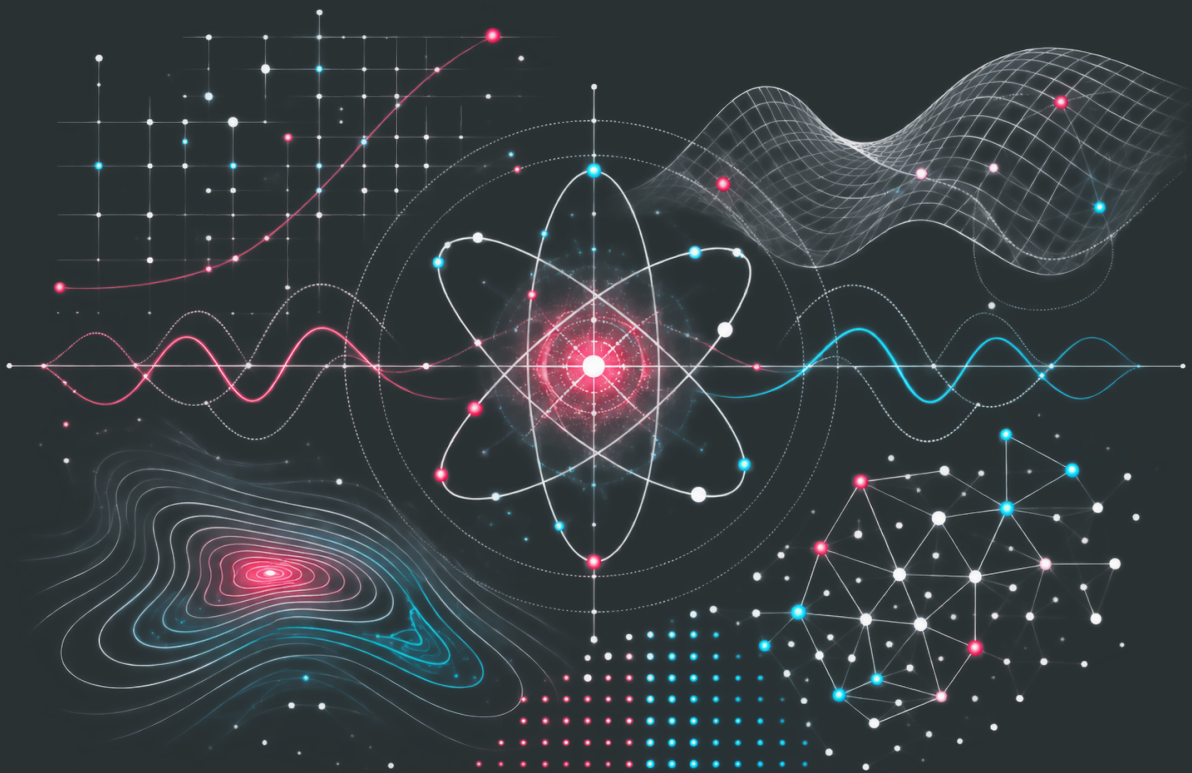
Новосибирск, 2026

Оглавление

Семестр 1. Последний рубеж	2
0 Вспомогательная литература	3
1 Системы счисления: основные обозначения	3
2 Десятичная система счисления	3
3 Перевод $q_1 \rightarrow q_2$	5
4 Дополнительные материалы	9
1 Представление чисел в ЭВМ	10
1 Целые числа	10
2 Числа с плавающей точкой	11
2 Решение нелинейных уравнений	17
1 Постановка задачи	17
2 Изоляция корня	18
3 Метод дихотомии	19
4 Метод простых итераций	22
5 Метод Ньютона	27
6 Метод секущих	30

Семестр 1

Последний рубеж



Вспомогательная литература

1 Системы счисления: основные обозначения

Число в системе с основанием q :

$$(a_n a_{n-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots a_{-m})_q.$$

$$(a_n \dots a_0 . a_{-1} \dots a_{-m})_q = \sum_{k=-m}^n a_k q^k, \quad 0 \leq a_k < q.$$

Основание	Система	Цифры	Пример
2	двоичная	0, 1	101101 ₂
8	восьмеричная	0, ..., 7	572 ₈
10	десятичная	0, ..., 9	935 ₁₀
16	шестнадцатеричная	0, ..., 9, A, ..., F	D6 ₁₆

$$A = 10, \quad B = 11, \quad C = 12, \quad D = 13, \quad E = 14, \quad F = 15.$$

2 Десятичная система счисления

Перевод $q \rightarrow 10$

Каждая цифра умножается на соответствующую степень основания:

$$(a_n \dots a_0 . a_{-1} \dots a_{-m})_q = a_n q^n + \dots + a_1 q + a_0 + a_{-1} q^{-1} + \dots + a_{-m} q^{-m}.$$

$2 \rightarrow 10$

$$101101.01_2$$

$$= 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2}$$

$$= 32 + 8 + 4 + 1 + \frac{1}{4} = 45.25_{10}.$$

$8 \rightarrow 10$

$$572_8 = 5 \cdot 8^2 + 7 \cdot 8 + 2$$

$$= 320 + 56 + 2 = 378_{10}.$$

$16 \rightarrow 10$

$$3A7_{16} = 3 \cdot 16^2 + 10 \cdot 16 + 7$$

$$= 768 + 160 + 7 = 935_{10}.$$

Перевод $10 \rightarrow q$

Целая часть

Последовательное деление на q :

$$\begin{aligned} N &= qN_1 + r_0, \\ N_1 &= qN_2 + r_1, \\ N_2 &= qN_3 + r_2, \\ &\vdots \end{aligned} \quad 0 \leq r_i < q.$$

Результат:

$$N_{10} = (r_k r_{k-1} \dots r_1 r_0)_q.$$

Остатки читаются снизу вверх.

Пример:

$$45_{10} \longrightarrow ?_2$$

$$\begin{aligned} 45 &= 2 \cdot 22 + 1, \\ 22 &= 2 \cdot 11 + 0, \\ 11 &= 2 \cdot 5 + 1, \\ 5 &= 2 \cdot 2 + 1, \\ 2 &= 2 \cdot 1 + 0, \\ 1 &= 2 \cdot 0 + 1. \end{aligned}$$

$$45_{10} = 101101_2.$$

Дробная часть

Последовательное умножение на q :

$$qx_i = a_{-(i+1)} + x_{i+1}, \quad 0 \leq x_{i+1} < 1.$$

Целые части читаются сверху вниз.

Пример:

$$0.625_{10} \longrightarrow ?_2$$

$$\begin{aligned} 0.625 \cdot 2 &= 1.25, \\ 0.25 \cdot 2 &= 0.50, \\ 0.50 \cdot 2 &= 1.00. \end{aligned}$$

$$0.625_{10} = 0.101_2.$$

Целая и дробная части

Переводятся отдельно:

$$45.625_{10} = 45_{10} + 0.625_{10}.$$

$$45_{10} = 101101_2, \quad 0.625_{10} = 0.101_2.$$

$$45.625_{10} = 101101.101_2.$$

3 Перевод $q_1 \rightarrow q_2$

Перевод $2 \rightarrow 8$

Разбить двоичное число на группы по 3 бита, начиная от точки.

$$\begin{array}{cccccccc} 000 & 001 & 010 & 011 & 100 & 101 & 110 & 111 \\ 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \end{array}$$

Целое число

$$11010110_2$$

Дополняем слева нулём:

$$011 \ 010 \ 110$$

$$011 \longrightarrow 3, \quad 010 \longrightarrow 2, \quad 110 \longrightarrow 6.$$

$$11010110_2 = 326_8.$$

Число с дробной частью

Группировка идёт от точки в обе стороны:

$$10110.1011_2$$

$$010 \ 110 \ . \ 101 \ 100$$

$$010 \longrightarrow 2, \quad 110 \longrightarrow 6,$$

$$101 \longrightarrow 5, \quad 100 \longrightarrow 4.$$

$$10110.1011_2 = 26.54_8.$$

Перевод $8 \rightarrow 2$

Каждую восьмеричную цифру заменить соответствующей тройкой битов:

0	1	2	3	4	5	6	7
000	001	010	011	100	101	110	111

Пример:

$$572_8$$

$$5 \longrightarrow 101, \quad 7 \longrightarrow 111, \quad 2 \longrightarrow 010.$$

$$572_8 = 101\ 111\ 010_2.$$

$$572_8 = 101111010_2.$$

Перевод $2 \rightarrow 16$

Разбить двоичное число на группы по 4 бита, начиная от точки.

0000	0001	0010	0011	0100	0101	0110	0111
0	1	2	3	4	5	6	7
1000	1001	1010	1011	1100	1101	1110	1111
8	9	A	B	C	D	E	F

Целое число

$$101111100011_2$$

$$1011\ 1110\ 0011$$

$$1011 \longrightarrow B, \quad 1110 \longrightarrow E, \quad 0011 \longrightarrow 3.$$

$$101111100011_2 = BE3_{16}.$$

Число с дробной частью

$$101101.1101_2$$

$$0010\ 1101 . 1101$$

$$0010 \longrightarrow 2, \quad 1101 \longrightarrow D.$$

$$101101.1101_2 = 2D.D_{16}.$$

Перевод $16 \rightarrow 2$

Каждую шестнадцатеричную цифру заменить соответствующей четвёркой битов.

0	1	2	3	4	5	6	7
0000	0001	0010	0011	0100	0101	0110	0111
8	9	A	B	C	D	E	F
1000	1001	1010	1011	1100	1101	1110	1111

Пример:

$$D6_{16}$$

$$D \longrightarrow 1101, \quad 6 \longrightarrow 0110.$$

$$D6_{16} = 1101\ 0110_2.$$

$$D6_{16} = 11010110_2.$$

Перевод $8 \leftrightarrow 16$

Перевод выполняется через двоичную систему.

$$8 \rightarrow 16$$

$$326_8$$

Сначала:

$$3 \longrightarrow 011, \quad 2 \longrightarrow 010, \quad 6 \longrightarrow 110.$$

$$326_8 = 011\ 010\ 110_2.$$

Перегруппируем по 4 бита:

$$011010110_2 = 0000\ 1101\ 0110_2.$$

4 Дополнительные материалы

Быстрая таблица перевода

	10	2	8	16
0	0000	0	0	
1	0001	1	1	
2	0010	2	2	
3	0011	3	3	
4	0100	4	4	
5	0101	5	5	
6	0110	6	6	
7	0111	7	7	
8	1000	10	8	
9	1001	11	9	
10	1010	12	A	
11	1011	13	B	
12	1100	14	C	
13	1101	15	D	
14	1110	16	E	
15	1111	17	F	

Краткий алгоритм выбора перевода

- $q \rightarrow 10$: $\sum_k a_k q^k$,
- $10 \rightarrow q$: деление / умножение на q ,
- $2 \rightarrow 8$: группы по 3 бита,
- $8 \rightarrow 2$: каждая цифра $\rightarrow$ 3 бита,
- $2 \rightarrow 16$: группы по 4 бита,
- $16 \rightarrow 2$: каждая цифра $\rightarrow$ 4 бита,
- $8 \leftrightarrow 16$: через двоичную систему.

Полезные степени двойки

n	0	1	2	3	4	5	6	7	8
2^n	1	2	4	8	16	32	64	128	256
n	9	10	11	12	13	14	15	16	
2^n	512	1024	2048	4096	8192	16384	32768	65536	

$$2^{10} = 1024 \approx 10^3, \quad 2^{20} \approx 10^6, \quad 2^{30} \approx 10^9.$$

Представление чисел в ЭВМ

Двоичная арифметика и машинная точность

1 Целые числа

Двоичное кодирование

В ЭВМ вся информация хранится в виде последовательности **битов**. Каждый бит может принимать только два значения:

$$0 \quad \text{или} \quad 1.$$

Следовательно, с помощью p бит можно закодировать

$$2^p$$

различных последовательностей.

Чтобы получить из последовательности битов число, каждому биту приписывают свой **вес разряда**. В привычной десятичной системе веса разрядов справа налево равны $1, 10, 100, \dots$: например, $305 = 3 \cdot 10^2 + 0 \cdot 10^1 + 5 \cdot 10^0$. В двоичной системе основание равно 2, поэтому веса разрядов справа налево равны $1, 2, 4, 8, \dots$, то есть степеням двойки.

Рассмотрим последовательность из p битов

$$a_{p-1}a_{p-2} \dots a_1a_0, \quad a_i \in \{0, 1\}.$$

Здесь a_i — значение отдельного бита, а i — номер его разряда. Разряды нумеруются **справа налево, начиная с нуля**: a_0 — самый правый бит с весом $2^0 = 1$, a_1 — следующий с весом $2^1 = 2$, и так далее; a_{p-1} — самый левый бит с весом 2^{p-1} . Индекс последнего разряда равен $p-1$, поскольку всего битов p , а отсчёт начинается с нуля. Запись $a_{p-1} \dots a_0$ означает цифры, поставленные рядом, а не их произведение.

Каждый бит вносит в значение числа слагаемое $a_i 2^i$: если $a_i = 1$, вес этого разряда прибавляется к сумме; если $a_i = 0$, вклад равен нулю. Поэтому значение двоичного числа равно

$$A = a_{p-1}2^{p-1} + a_{p-2}2^{p-2} + \dots + a_12 + a_0 = \sum_{i=0}^{p-1} a_i 2^i.$$

Знак $\sum_{i=0}^{p-1}$ кратко обозначает сложение вкладов всех разрядов: нужно подставить $i = 0, 1, \dots, p-1$ и сложить полученные слагаемые.

Например, для четырёх битов 1011 имеем $a_3 = 1$, $a_2 = 0$, $a_1 = 1$, $a_0 = 1$. Тогда

$$(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = 11.$$

Индекс 2 у скобок указывает на двоичную запись; справа получено значение в десятичной системе.

Таким способом кодируются беззнаковые целые числа

$$0, 1, \dots, 2^p - 1.$$

Например, для $p = 32$ имеется

$$2^{32}$$

различных битовых последовательностей.

Для стандартного 32-битного целого числа

$$4 \text{ байта} \times 8 \text{ бит} = 32 \text{ бита}.$$

Отрицательные числа

Для представления целых чисел со знаком необходимо закодировать как положительные, так и отрицательные значения.

В дополнительном коде отрицательное число $-N$ представляется числом

$$2^p - N.$$

Иными словами, код числа $-N$ выбирается так, чтобы

$$N + (2^p - N) = 2^p \equiv 0 \pmod{2^p}.$$

Поэтому арифметику p -битных целых чисел удобно рассматривать как арифметику по модулю 2^p .

Например,

$$-1 \longleftrightarrow 2^p - 1 = \underbrace{11 \dots 11}_{p \text{ бит}}_2.$$

Диапазон знаковых p -битных целых чисел имеет вид

$$-2^{p-1} \leq N \leq 2^{p-1} - 1.$$

Замечание. Именно поэтому при переполнении целочисленной переменной результат фактически “переходит через границу” доступного диапазона.

2 Числа с плавающей точкой

Фиксированная и плавающая точка

Вещественное число можно хранить в форме с **фиксированной точкой**, заранее задавая положение десятичного разделителя.

Например,

$$1,031 = \frac{1031}{1000}, \quad 10,05 = \frac{1005}{100}.$$

Такой способ неудобен для чисел, изменяющихся в очень широком диапазоне. Поэтому на практике используется **форма с плавающей точкой**:

$$x = M \cdot b^E,$$

где M — мантисса, E — порядок, а b — основание системы счисления.

Например, в десятичной системе

$$1,031 = 1,031 \cdot 10^0,$$

$$10,51 = 1,051 \cdot 10^1.$$

Нормализованная форма

Чтобы запись числа с плавающей точкой была однозначной, используется **нормализованная форма**: положение запятой выбирается так, чтобы перед ней находилась ровно одна ненулевая цифра.

Например,

$$1,031 \cdot 10^0$$

является нормализованной записью, тогда как

$$10,31 \cdot 10^{-1}$$

описывает то же число, но нормализованной записью не является.

В двоичной системе первая цифра нормализованной мантиссы всегда равна единице. Поэтому число можно записать в форме

$$x = (-1)^S \left(1 + \frac{M}{2^n} \right) 2^E,$$

где

$$S \in \{0, 1\}, \quad M = 0, 1, \dots, 2^n - 1.$$

Здесь

- S — бит знака;
- M — целое число, кодирующее дробную часть мантиссы;
- n — число бит дробной части мантиссы;
- E — порядок.

Поскольку старшая единица нормализованной мантиссы всегда известна, её нет необходимости явно хранить в памяти. Это так называемая **скрытая единица**.

Формат IEEE 754

В памяти нормализованное число записывается в виде трёх полей:

$$\boxed{S} \quad \boxed{E_S} \quad \boxed{M},$$

где S — знак, E_S — смещённый порядок, M — дробная часть мантиссы.

Если под порядок выделено w бит, вводится смещение

$$B = 2^{w-1} - 1.$$

Тогда вместо самого порядка E хранится

$$E_S = E + B = E + 2^{w-1} - 1.$$

Следовательно,

$$E = E_S - (2^{w-1} - 1),$$

и нормализованное число принимает вид

$$x = (-1)^S \left(1 + \frac{M}{2^n}\right) 2^{E_S - (2^{w-1} - 1)}.$$

Два значения поля порядка зарезервированы для специальных случаев:

$$E_S = 0, \quad E_S = 2^w - 1.$$

Поэтому для нормализованных чисел

$$1 \leq E_S \leq 2^w - 2.$$

Подставляя

$$E_S = E + 2^{w-1} - 1,$$

получаем

$$\begin{aligned} 1 &\leq E + 2^{w-1} - 1 \leq 2^w - 2, \\ -2^{w-1} + 2 &\leq E \leq 2^{w-1} - 1. \end{aligned}$$

Таким образом,

$$E_{\min} = -2^{w-1} + 2, \quad E_{\max} = 2^{w-1} - 1.$$

Одинарная точность: float

Для формата IEEE 754 binary32, которому обычно соответствует тип `float`,

$$32 = 1 + 8 + 23.$$

Биты распределяются следующим образом:

$$\underbrace{\boxed{S}}_{1 \text{ бит}} \quad \underbrace{\boxed{E_S}}_{8 \text{ бит}} \quad \underbrace{\boxed{M}}_{23 \text{ бита}}.$$

То есть

$$w = 8, \quad n = 23, \quad B = 2^7 - 1 = 127.$$

Отсюда

$$E_{\min} = -126, \quad E_{\max} = 127.$$

Минимальное положительное нормализованное число получается при $M = 0$ и $E = E_{\min}$:

$$f_0 = 2^{E_{\min}} = 2^{-126} \approx 1.18 \cdot 10^{-38}.$$

Следующее машинное число имеет $M = 1$:

$$f_1 = \left(1 + \frac{1}{2^n}\right) 2^{E_{\min}}.$$

Расстояние между ними равно

$$f_1 - f_0 = 2^{E_{\min} - n}.$$

При этом

$$\frac{f_0}{f_1 - f_0} = 2^n.$$

Для `float`, где $n = 23$,

$$2^{23} \approx 8.4 \cdot 10^6.$$

То есть без дополнительного механизма около нуля возникла бы очень большая область непредставимых чисел.

Денормализованные числа

Чтобы заполнить область около нуля, вводятся **денормализованные** (subnormal) числа.

Для них поле порядка полностью заполнено нулями:

$$E_S = 0.$$

Скрытая единица в мантиссе при этом отсутствует, поэтому число записывается как

$$x = (-1)^S \frac{M}{2^n} 2^{E_{\min}}.$$

Минимальное положительное денормализованное число соответствует $M = 1$:

$$f_{\min}^{\text{denorm}} = 2^{E_{\min} - n}.$$

Максимальное денормализованное число:

$$f_{\max}^{\text{denorm}} = \left(1 - \frac{1}{2^n}\right) 2^{E_{\min}}.$$

А следующее за ним число уже является минимальным нормализованным:

$$f_{\min}^{\text{norm}} = 2^{E_{\min}}.$$

Таким образом, переход от денормализованных чисел к нормализованным происходит без большого разрыва.

Специальные значения

Зарезервированные значения поля порядка позволяют кодировать ноль, бесконечность и NaN.

Для краткости обозначим через $00 \dots 0$ последовательность из нулей, а через $11 \dots 1$ — последовательность из единиц.

1.

$$\boxed{S} \boxed{00 \dots 0} \boxed{00 \dots 0}$$

соответствует числу $+0$ или -0 в зависимости от S .

2.

$$\boxed{S} \boxed{00 \dots 0} \boxed{M}, \quad M \neq 0,$$

соответствует денормализованному числу.

3.

$$\boxed{S} \boxed{11 \dots 1} \boxed{00 \dots 0}$$

соответствует

$$+\infty \quad \text{или} \quad -\infty.$$

4.

$$\boxed{S} \boxed{11 \dots 1} \boxed{M}, \quad M \neq 0,$$

соответствует NaN (*Not a Number*).

5. При

$$1 \leq E_S \leq 2^w - 2$$

кодируется обычное нормализованное число.

Машинная точность

Поскольку число бит мантииссы конечно, результат арифметической операции в общем случае приходится округлять до ближайшего представимого машинного числа.

Например,

$$1,08 \cdot 2,03 = 2,1924.$$

Если оставить только три значащие цифры, получим

$$2,1924 \longrightarrow 2,19,$$

что вносит относительную погрешность порядка

$$10^{-3}.$$

Рассмотрим теперь машинные числа около единицы. При $E = 0$ они имеют вид

$$f = 1 + \frac{M}{2^n}.$$

Два соседних числа:

$$f_0 = 1, \quad f_1 = 1 + \frac{1}{2^n}.$$

Расстояние между ними равно

$$f_1 - f_0 = \frac{1}{2^n}.$$

Эта величина называется **машинным эпсилоном**:

$$\boxed{\varepsilon = 2^{-n}}.$$

Она характеризует относительное расстояние между соседними машинными числами порядка единицы.

При округлении к ближайшему числу максимальная относительная погрешность имеет порядок

$$\frac{\varepsilon}{2} = 2^{-(n+1)}.$$

Практически машинное эпсилон можно характеризовать условиями

$$\begin{aligned} 1 + \varepsilon &\neq 1, \\ 1 + \frac{\varepsilon}{2} &= 1 \quad \text{в машинной арифметике.} \end{aligned}$$

Для одинарной точности

$$\varepsilon_{\text{float}} = 2^{-23} \approx 1.19 \cdot 10^{-7},$$

а для двойной точности

$$\varepsilon_{\text{double}} = 2^{-52} \approx 2.22 \cdot 10^{-16}.$$

Решение нелинейных уравнений

Методы дихотомии, простых итераций и Ньютона

1 Постановка задачи

Будем рассматривать численное решение нелинейного уравнения одной вещественной переменной

$$f(x) = 0.$$

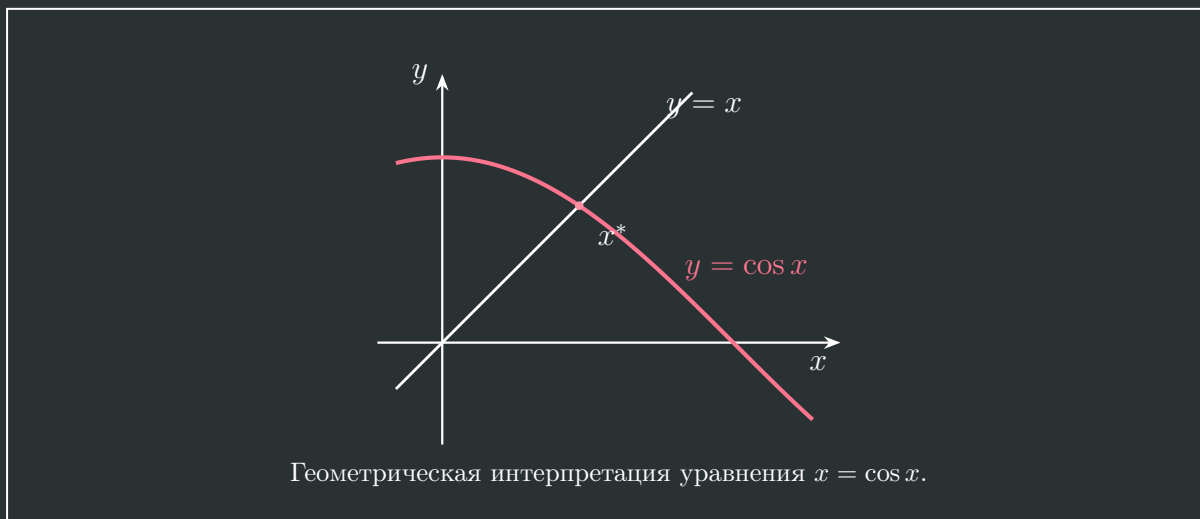
В общем случае такое уравнение не удаётся решить аналитически. Простейший пример — уравнение

$$x = \cos x,$$

которое можно переписать в стандартном виде

$$x - \cos x = 0.$$

Геометрически его корень определяется как точка пересечения графиков $y = x$ и $y = \cos x$.



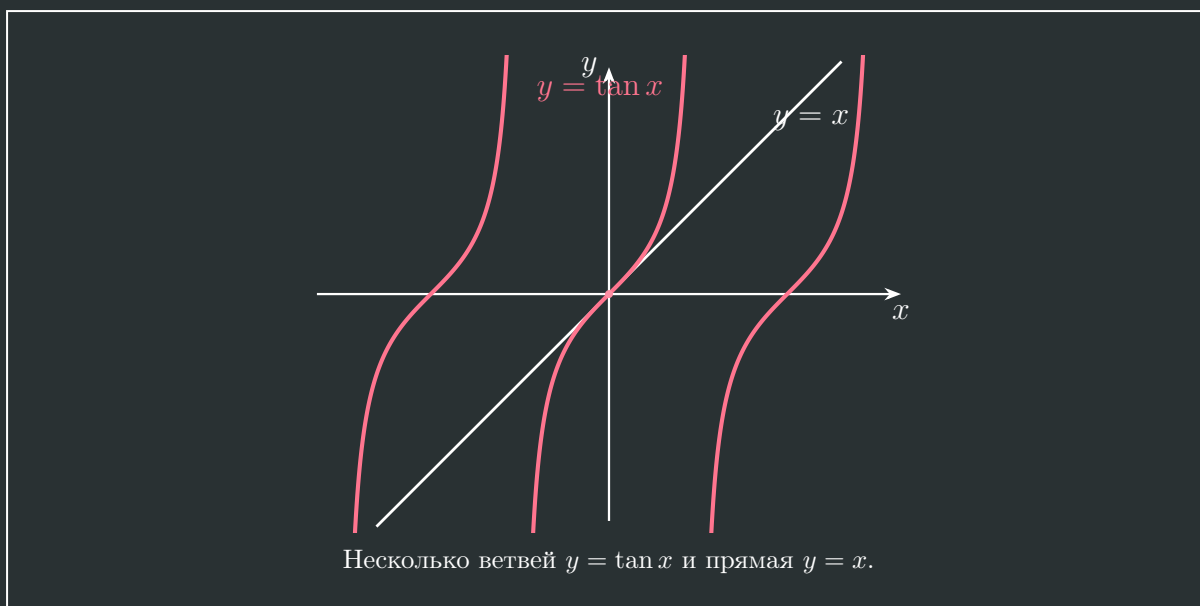
Аналогично уравнение

$$x = \tan x$$

можно записать как

$$x - \tan x = 0.$$

В отличие от предыдущего примера, такое уравнение имеет несколько вещественных корней.



Таким образом, прежде чем применять численный метод, необходимо выбрать интересующий нас корень и локализовать его на некотором отрезке.

2 Изоляция корня

Пусть требуется найти корень x^* уравнения

$$f(x) = 0.$$

Найдём интервал

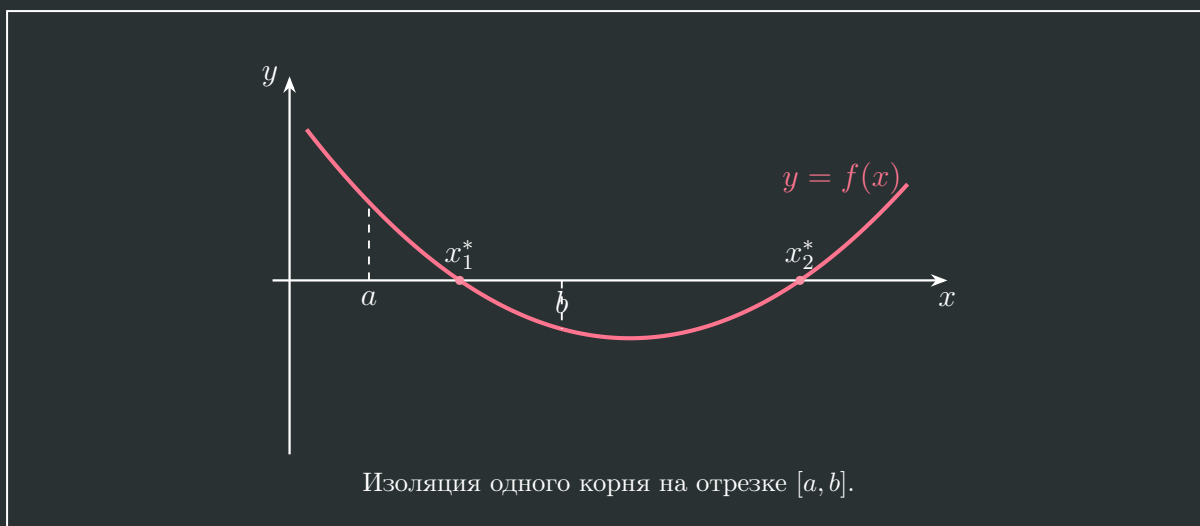
$$[a, b],$$

содержащий интересующий нас корень. Будем считать, что на этом интервале выполняются следующие условия:

- 1) на $[a, b]$ находится только один корень x^* ;
- 2) функция $f(x)$ монотонна на $[a, b]$;
- 3) функция $f(x)$ непрерывна на $[a, b]$;
- 4) значения функции на концах имеют разные знаки либо один из концов сам является корнем:

$$f(a)f(b) \leq 0.$$

Для непрерывной функции последнее условие гарантирует существование хотя бы одного корня внутри отрезка. Дополнительное условие единственности позволяет говорить именно об изолированном корне.



3 Метод дихотомии

Построение последовательности отрезков

Пусть первоначально

$$a_0 = a, \quad b_0 = b,$$

и

$$f(a_0)f(b_0) \leq 0.$$

На первом шаге делим исходный отрезок пополам и вычисляем его середину:

$$x_1 = \frac{a_0 + b_0}{2}.$$

Если

$$f(a_0)f(x_1) \leq 0,$$

то выбираем левую половину:

$$a_1 = a_0, \quad b_1 = x_1.$$

Если же

$$f(a_0)f(x_1) > 0,$$

то корень находится в правой половине и полагаем

$$a_1 = x_1, \quad b_1 = b_0.$$

В обоих случаях сохраняется условие

$$f(a_1)f(b_1) \leq 0,$$

а ширина нового интервала в два раза меньше исходной:

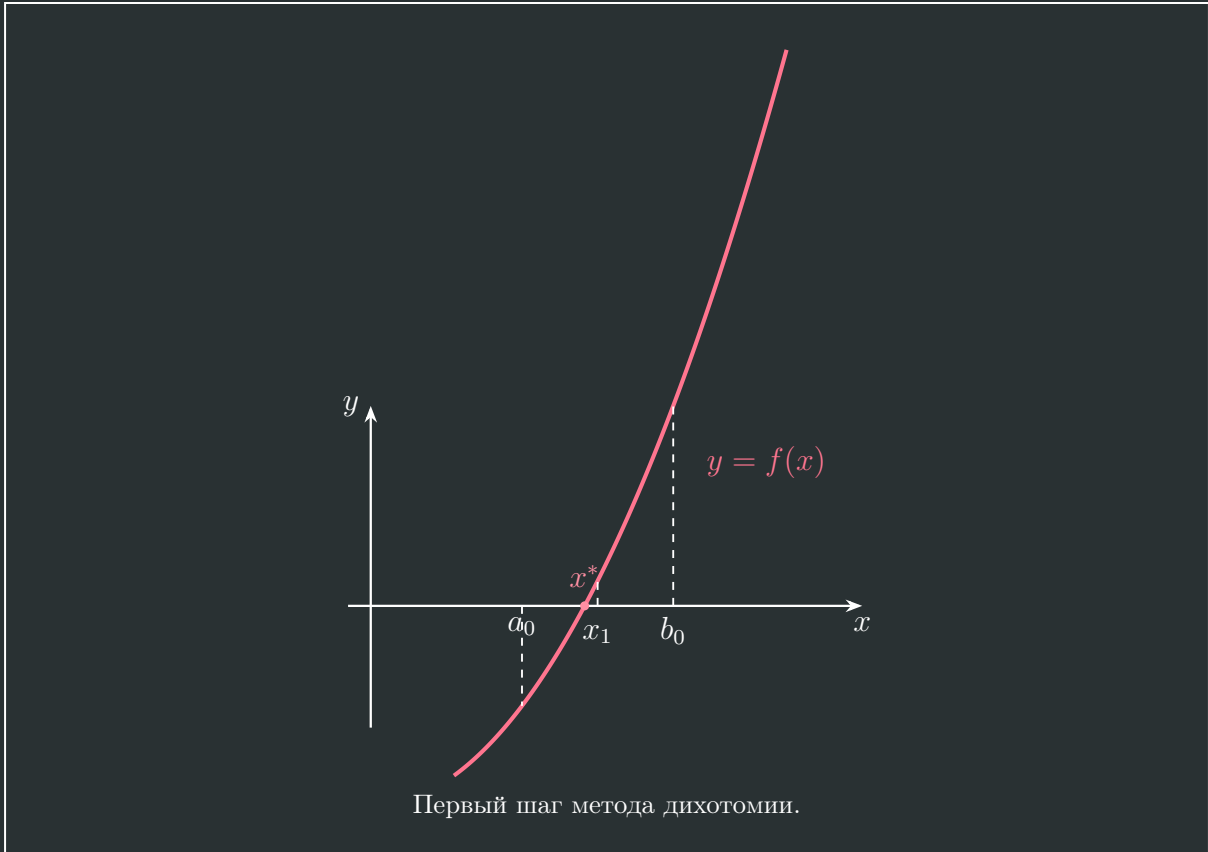
$$b_1 - a_1 = \frac{b_0 - a_0}{2}.$$

Введём обозначение

$$\delta_0 = b_0 - a_0.$$

Тогда

$$\delta_1 = \frac{\delta_0}{2}.$$



На втором шаге повторяем ту же процедуру уже для отрезка $[a_1, b_1]$:

$$x_2 = \frac{a_1 + b_1}{2}.$$

После выбора нужной половины получаем

$$b_2 - a_2 = \delta_2 = \frac{b_0 - a_0}{2^2} = \frac{\delta_0}{2^2}.$$

Продолжая процесс, на n -м шаге имеем

$$x_n = \frac{a_{n-1} + b_{n-1}}{2},$$

а ширина нового интервала равна

$$\begin{aligned} b_n - a_n &= \frac{b_{n-1} - a_{n-1}}{2} \\ &= \frac{b_0 - a_0}{2^n} \\ &= \frac{\delta_0}{2^n}. \end{aligned}$$

Таким образом,

$$\delta_n = \frac{\delta_0}{2^n}.$$

Число шагов

Условием останова итерационного процесса можно взять достижение заданной точности

$$\delta_n \leq \varepsilon.$$

Тогда

$$\frac{\delta_0}{2^N} \leq \varepsilon,$$

откуда

$$2^N \geq \frac{\delta_0}{\varepsilon}.$$

Следовательно, необходимое число шагов оценивается как

$$N \approx \log_2 \frac{\delta_0}{\varepsilon} = \frac{\ln(\delta_0/\varepsilon)}{\ln 2}.$$

Если в качестве приближения к корню брать середину текущего интервала, то максимальная абсолютная погрешность вдвое меньше его ширины:

$$|x_n - x^*| \leq \frac{b_{n-1} - a_{n-1}}{2}.$$

Сходимость метода

Последовательность левых границ

$$\{a_n\}$$

монотонно возрастает и ограничена сверху. Последовательность правых границ

$$\{b_n\}$$

монотонно убывает и ограничена снизу. Поэтому обе последовательности имеют пределы:

$$a_n \rightarrow A, \quad b_n \rightarrow B.$$

Но

$$b_n - a_n = \frac{\delta_0}{2^n} \xrightarrow{n \rightarrow \infty} 0,$$

следовательно,

$$A = B = C.$$

На каждом шаге сохраняется условие

$$f(a_n)f(b_n) \leq 0.$$

Переходя к пределу и используя непрерывность функции $f(x)$, получаем

$$f(C)f(C) \leq 0.$$

То есть

$$[f(C)]^2 \leq 0.$$

Поскольку квадрат вещественного числа неотрицателен,

$$f(C) = 0.$$

Следовательно,

$$C = x^*,$$

и

$$x_n = \frac{a_{n-1} + b_{n-1}}{2} \xrightarrow{n \rightarrow \infty} x^*.$$

Таким образом, метод дихотомии обладает гарантированной сходимостью для непрерывной функции при наличии изменения знака на концах отрезка. Недостаток метода — сравнительно невысокая скорость сходимости.

4 Метод простых итераций

Переход к уравнению на неподвижную точку

Исходное уравнение

$$f(x) = 0$$

перепишем в эквивалентном виде

$$x = \varphi(x).$$

Например, простейший вариант такого преобразования имеет вид

$$f(x) = 0 \iff x + f(x) = x,$$

то есть

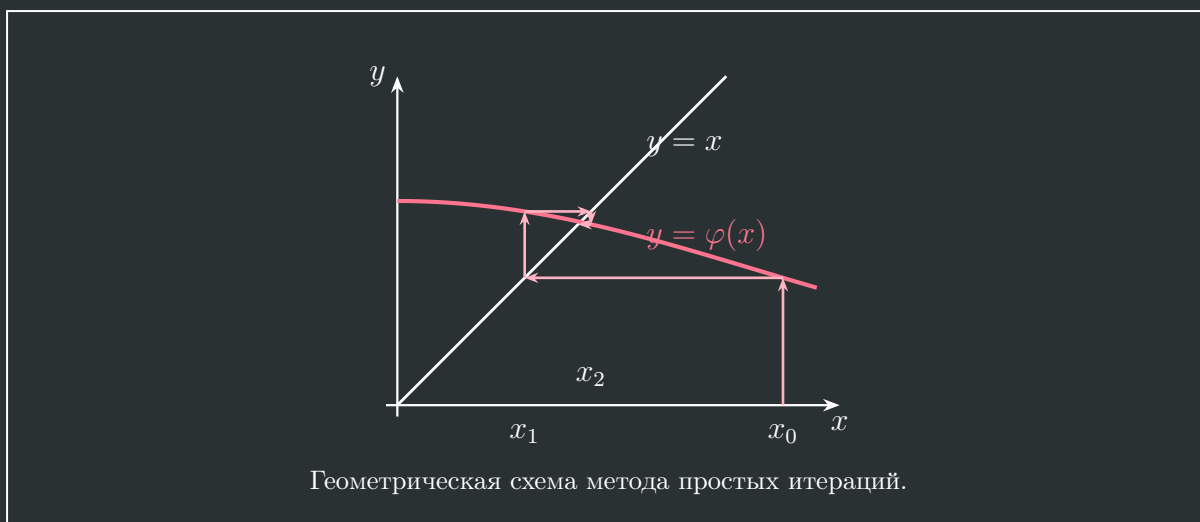
$$\varphi(x) = x + f(x).$$

После этого строим итерационный процесс

$$x_{n+1} = \varphi(x_n), \quad n = 0, 1, 2, \dots$$

Выбираем начальное приближение x_0 и последовательно вычисляем

$$x_1 = \varphi(x_0), \quad x_2 = \varphi(x_1), \quad x_3 = \varphi(x_2), \dots$$



Условие сходимости

Пусть $[a, b]$ — интервал, содержащий корень x^* , причём

$$\varphi([a, b]) \subset [a, b].$$

Пусть функция $\varphi(x)$ непрерывно дифференцируема на $[a, b]$ и существует число $q < 1$ такое, что

$$|\varphi'(x)| \leq q < 1, \quad x \in [a, b].$$

Тогда итерационный процесс

$$x_{n+1} = \varphi(x_n)$$

сходится к единственной неподвижной точке x^* на этом интервале.

Действительно, для корня

$$x^* = \varphi(x^*).$$

Поэтому

$$|x_n - x^*| = |\varphi(x_{n-1}) - \varphi(x^*)|.$$

По теореме Лагранжа существует точка ξ_{n-1} между x_{n-1} и x^* такая, что

$$\begin{aligned} |x_n - x^*| &= |\varphi'(\xi_{n-1})| |x_{n-1} - x^*| \\ &\leq q |x_{n-1} - x^*|. \end{aligned}$$

Для предыдущего шага аналогично

$$|x_{n-1} - x^*| \leq q |x_{n-2} - x^*|.$$

Следовательно,

$$\begin{aligned} |x_n - x^*| &\leq q^2 |x_{n-2} - x^*| \\ &\leq \dots \\ &\leq q^n |x_0 - x^*|. \end{aligned}$$

Так как

$$0 \leq q < 1,$$

то

$$q^n \xrightarrow{n \rightarrow \infty} 0,$$

и поэтому

$$x_n \xrightarrow{n \rightarrow \infty} x^*.$$

Сходящийся и расходящийся процессы

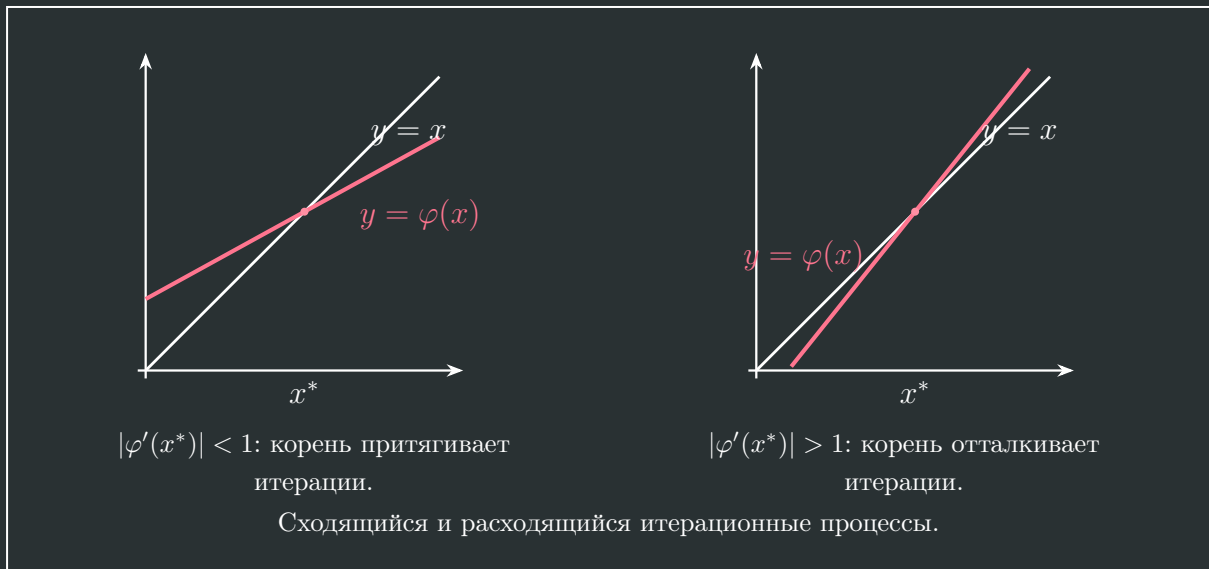
Условие

$$|\varphi'(x)| < 1$$

соответствует сжимающему отображению: расстояние до неподвижной точки уменьшается после каждой итерации. Если же в окрестности корня

$$|\varphi'(x^*)| > 1,$$

то неподвижная точка становится отталкивающей и итерации обычно расходятся.



Ограничение $|\varphi'| < 1$ является достаточным, но не необходимым условием сходимости. В более общем случае достаточно, чтобы само отображение уменьшало расстояние до искомой неподвижной точки.

Скорость сходимости

Пусть начальное приближение x_0 уже достаточно близко к корню. Разложим $\varphi(x_0)$ около x^* :

$$\varphi(x_0) \approx \varphi(x^*) + \varphi'(x^*)(x_0 - x^*).$$

Так как

$$\varphi(x^*) = x^*,$$

то

$$x_1 - x^* \approx \varphi'(x^*)(x_0 - x^*).$$

Введём абсолютную погрешность

$$\delta_n = |x_n - x^*|.$$

Тогда

$$\delta_1 \approx |\varphi'(x^*)| \delta_0.$$

На следующем шаге

$$\delta_2 \approx |\varphi'(x^*)| \delta_1 \approx |\varphi'(x^*)|^2 \delta_0.$$

Продолжая,

$$\delta_n \approx |\varphi'(x^*)|^n \delta_0.$$

Если требуется достичь точности ε , то

$$\varepsilon \approx |\varphi'(x^*)|^N \delta_0.$$

Следовательно,

$$N \approx \frac{\ln(\delta_0/\varepsilon)}{-\ln |\varphi'(x^*)|}.$$

Сравнивая эту оценку с методом дихотомии, получаем, что метод простых итераций сходится быстрее при

$$|\varphi'(x^*)| < \frac{1}{2}.$$

Особенно быстрой будет сходимость при

$$\varphi'(x^*) = 0,$$

однако в этом случае линейная оценка уже недостаточна и необходимо учитывать следующие члены разложения Тейлора.

Переход к обратной функции

Если

$$|\varphi'(x)| > 1,$$

то прямой итерационный процесс

$$x_{n+1} = \varphi(x_n)$$

может расходиться. Если обратную функцию удобно вычислять, можно перейти от

$$x = \varphi(x)$$

к эквивалентному уравнению

$$x = \varphi^{-1}(x).$$

При этом

$$(\varphi^{-1})'(x) = \frac{1}{\varphi'(\varphi^{-1}(x))}.$$

В окрестности неподвижной точки

$$|(\varphi^{-1})'(x^*)| = \frac{1}{|\varphi'(x^*)|} < 1,$$

так что итерации для обратной функции становятся сходящимися.

Корректирующий множитель

Вместо простейшего представления

$$x = x + f(x)$$

можно использовать эквивалентное уравнение

$$x = x - \lambda f(x), \quad \lambda \neq 0.$$

Определим

$$\tilde{\varphi}(x) = x - \lambda f(x).$$

Тогда

$$\tilde{\varphi}'(x) = 1 - \lambda f'(x).$$

Для сходимости достаточно потребовать

$$|1 - \lambda f'(x)| < 1.$$

Раскрывая это неравенство,

$$\begin{aligned} -1 &< 1 - \lambda f'(x) < 1, \\ -2 &< -\lambda f'(x) < 0, \end{aligned}$$

то есть

$$0 < \lambda f'(x) < 2.$$

Поэтому знак λ следует выбирать совпадающим со знаком $f'(x)$. Если производная не меняет знак на всём рабочем интервале, то достаточное условие можно записать как

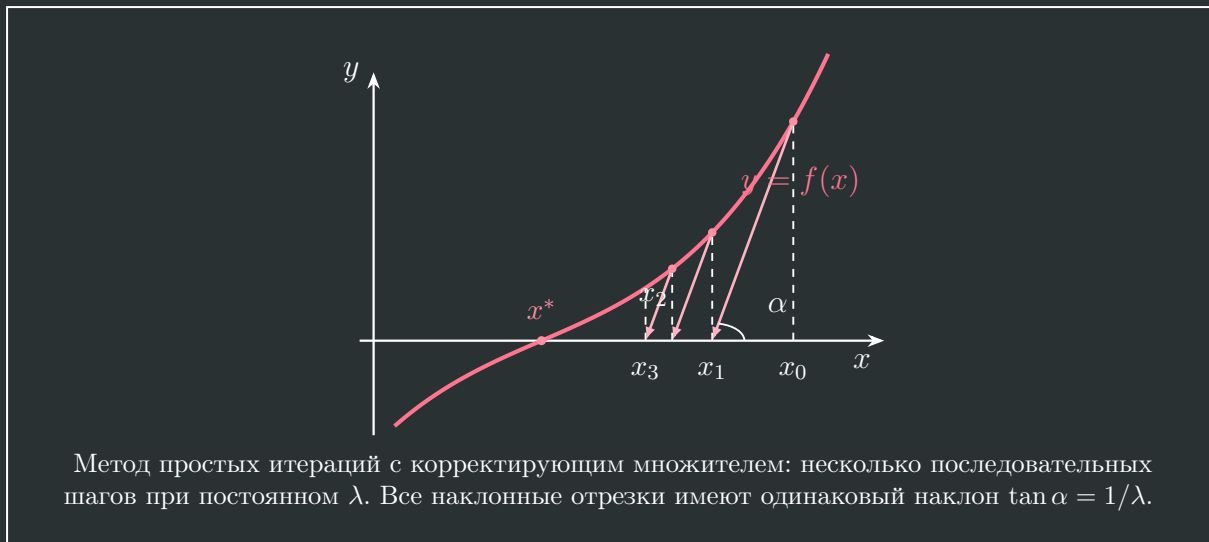
$$0 < |\lambda| < \frac{2}{\max_{x \in [a, b]} |f'(x)|}.$$

При этом для ускорения сходимости значение λ желательно выбирать как можно ближе к

$$\frac{1}{f'(x^*)}.$$

Итерационный процесс принимает вид

$$x_{n+1} = x_n - \lambda f(x_n).$$



Если угол прямой с осью x обозначить через α , то

$$\tan \alpha = \frac{1}{\lambda}.$$

Именно эта геометрическая интерпретация приводит к следующему ускорению метода: вместо прямой с постоянным наклоном будем на каждом шаге использовать касательную к графику $y = f(x)$.

5 Метод Ньютона

Формула метода

Для касательной к графику $y = f(x)$ в точке x_n имеем

$$\tan \alpha = f'(x_n).$$

Сравнивая это с

$$\tan \alpha = \frac{1}{\lambda},$$

выбираем

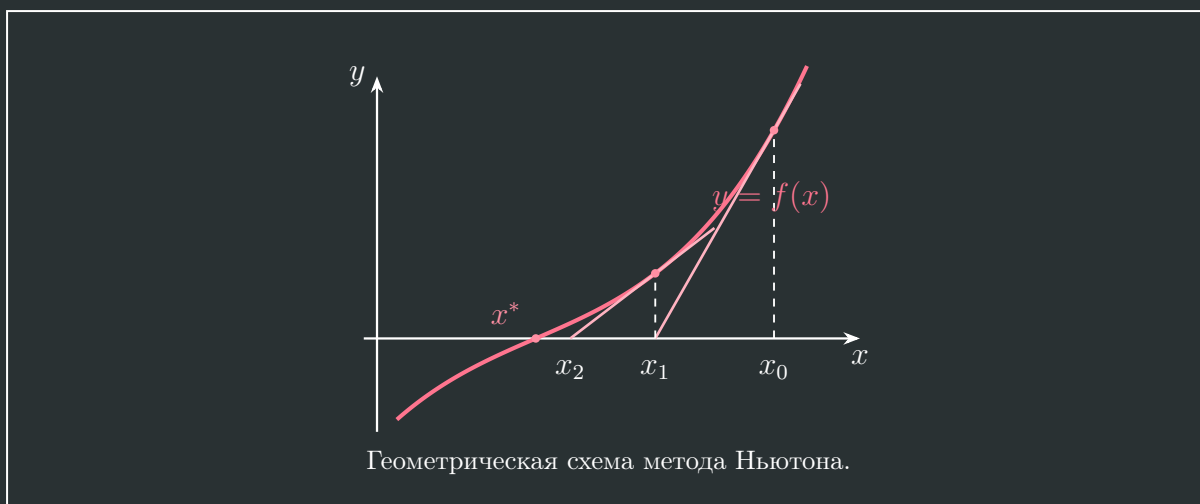
$$\lambda(x_n) = \frac{1}{f'(x_n)}.$$

Тогда итерационный процесс принимает вид

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Это **метод Ньютона**, или метод Ньютона–Рафсона.

Геометрически на каждом шаге проводим касательную к графику $y = f(x)$ в точке $(x_n, f(x_n))$. Точка пересечения этой касательной с осью x даёт следующее приближение x_{n+1} .



Квадратичная сходимость

Оценим погрешность вблизи простого корня x^* , для которого

$$f(x^*) = 0, \quad f'(x^*) \neq 0.$$

Введём знаковую погрешность

$$R_n = x^* - x_n.$$

Тогда

$$x_n - x^* = -R_n.$$

Из формулы Ньютона

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

получаем

$$\begin{aligned} R_{n+1} &= x^* - x_{n+1} \\ &= x^* - x_n + \frac{f(x_n)}{f'(x_n)} \\ &= R_n + \frac{f(x_n)}{f'(x_n)}. \end{aligned}$$

Разложим функцию около корня:

$$\begin{aligned} f(x_n) &\approx f(x^*) + f'(x^*)(x_n - x^*) + \frac{1}{2}f''(x^*)(x_n - x^*)^2 \\ &= -f'(x^*)R_n + \frac{1}{2}f''(x^*)R_n^2. \end{aligned}$$

Аналогично для производной

$$\begin{aligned} f'(x_n) &\approx f'(x^*) + f''(x^*)(x_n - x^*) \\ &= f'(x^*) - f''(x^*)R_n. \end{aligned}$$

Подставим эти выражения в формулу для R_{n+1} :

$$R_{n+1} \approx R_n + \frac{-f'(x^*)R_n + \frac{1}{2}f''(x^*)R_n^2}{f'(x^*) - f''(x^*)R_n}.$$

Приведём два слагаемых к общему знаменателю:

$$\begin{aligned} R_{n+1} &\approx \frac{R_n(f'(x^*) - f''(x^*)R_n) - f'(x^*)R_n + \frac{1}{2}f''(x^*)R_n^2}{f'(x^*) - f''(x^*)R_n} \\ &= \frac{-\frac{1}{2}f''(x^*)R_n^2}{f'(x^*) - f''(x^*)R_n}. \end{aligned}$$

Вынесем $f'(x^*)$ из знаменателя:

$$R_{n+1} \approx -\frac{1}{2} \frac{f''(x^*)}{f'(x^*)} \frac{R_n^2}{1 - \frac{f''(x^*)}{f'(x^*)}R_n}.$$

Вблизи корня $|R_n|$ мало, поэтому

$$\left| \frac{f''(x^*)}{f'(x^*)} R_n \right| \ll 1.$$

Следовательно,

$$R_{n+1} \approx -\frac{1}{2} \frac{f''(x^*)}{f'(x^*)} R_n^2.$$

Для абсолютной погрешности

$$\delta_n = |R_n|$$

получаем

$$\delta_{n+1} \approx \frac{1}{2} \left| \frac{f''(x^*)}{f'(x^*)} \right| \delta_n^2.$$

Таким образом, погрешность следующего приближения пропорциональна квадрату предыдущей погрешности:

$$\delta_{n+1} \sim \delta_n^2.$$

Поэтому метод Ньютона обладает **квадратичной сходимостью**. При достаточно хорошем начальном приближении число верных знаков примерно удваивается после каждой итерации.

Недостатки метода

Метод Ньютона очень быстро сходится вблизи простого корня, но имеет два существенных недостатка.

Во-первых, он чувствителен к начальному приближению. Если x_0 выбрано вне области притяжения искомого корня, процесс может расходиться либо сойтись к другому корню.

Во-вторых, на каждом шаге необходимо вычислять производную

$$f'(x_n).$$

Последний недостаток приводит к методу секущих.

